Machine Learning to Approximate MaxSAT

作者：陳廣能、鍾陳祐
指導教授：林莊傑

目錄

摘要與簡介	創作心路歷程、研究動機
核心理論	理論推導、架構與演算法則
實驗結果	性能分析與比較
結論與展望	研究摘要、未來展望

Github Repository URL:
https://github.com/HilamJeff67/graph-sat/tree/main

摘要與簡介

創作心路歷程

Phase 01

研究 TorchmSAT
相關論文及其架構

Phase 02

嘗試 Backtrack
暴力求解 MaxSAT

Phase 03

準備 Dataset
（由 MaxSAT Benchmark
以及 PySAT 提供）

Phase 04

研究並測試 LRU、
RC2 等傳統求解器

Phase 05

嘗試使用 Attention
優化 TorchmSAT

Phase 06

觀察出 Bipartite 結構
使用 Message Passing 機制
並優化 TorchmSAT

摘要與簡介

基礎名詞

CNF formula :

$$(x_1 \vee -x_2 \vee x_3) \wedge (-x_1 \vee x_2) \wedge (x_2 \vee -x_3).$$

x_1 , x_2 , x_3

變數 (Variables)

$(x_1 \vee -x_2 \vee x_3)$

$(-x_1 \vee x_2)$

$(x_2 \vee -x_3)$

子句 (Clauses)

摘要與簡介

傳統求解器與 CDCL (Conflict-driven clause learning) 演算法

傳統求解器(以下稱 Oracle), 利用 CDCL 演算法的大量剪枝跟 啟發式策略來暴力求解 MaxSAT 問題, 一些基於 Oracle 的演算法例如 LSU 透過暴力解出來的結果再去線性嘗試有沒有可能有更大可滿足的子句, 或是像 Core-Guided Binary Search 在嘗試更大可滿足子句時使用二分搜尋法, 這些雖然在 MaxSAT Benchmark 競賽中獲得很亮眼的表現, 但是仍然是在暴力求解, 我們嘗試使用 PySAT 提供的 hard 問題來對這些演算法測試, 這些 hard 問題通常都會讓這些演算法陷入剪枝困難或是生成更多多餘子句的情況, 使得最終運行了超過數小時都沒有結果, 而後面將會提到的 TorchmSAT 和 GraphSAT 都是僅在數秒至數分 內即可得到近似解, 基於機器學習的方式, 如果 Iteration 更多將會有更高機率探索到更好的解。

摘要與簡介

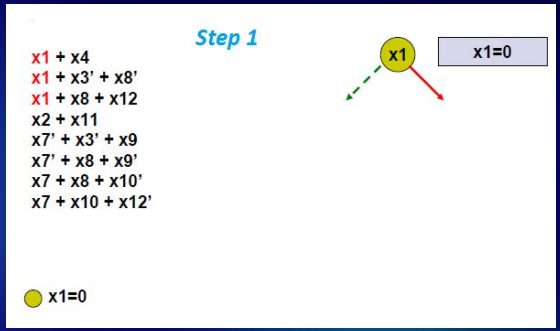
CDCL (Conflict-driven clause learning) 演算法

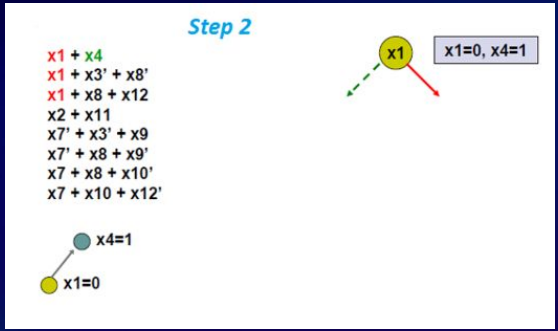
大致步驟如下:

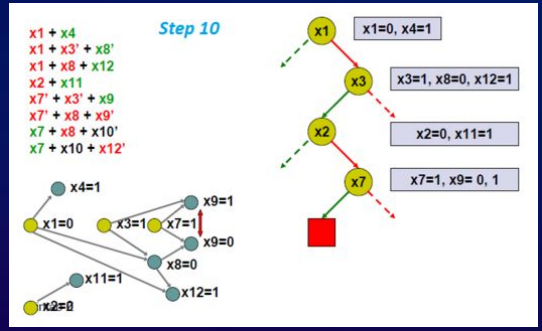
1. **變數決策與傳播（Decision & Unit Propagation)**:求解器會透過啟發式策略(例如 VSIDS, 優先挑選常參與衝突的變數)決定下一個要賦 值的變數。賦值後, 立刻執行 Unit Propagation, 強制指派那些因前面決策而「別無選擇」的相依變數(例如在 X_1 or X_2 中, 若 X_1 = False, 則必推導出 X_2 =True)。
2. **衝突分析（Conflict Analysis) 與學習（Clause Learning)**:當某個 Literal 被推導出必須同時為 True 和 False 時, 即產生衝突。演算法會分析導致衝突的「決策祖先」, 將這次錯誤經驗總結成一個新的子句加入原本的問題中。
3. **非循序回溯（Non-Chronological Backtracking)**:根據學習到的新子句, 直接跳躍回溯到引發錯誤的分支點, 避免浪費時間搜尋注定失敗的相似空間。透過不斷「學習」新子句, 未來能在更早的階段觸發 Unit Propagation, 大幅限縮搜尋空間。

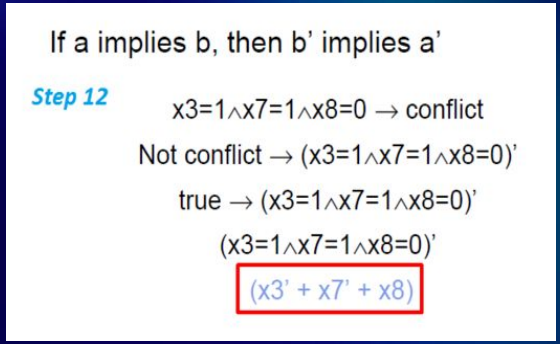
摘要與簡介

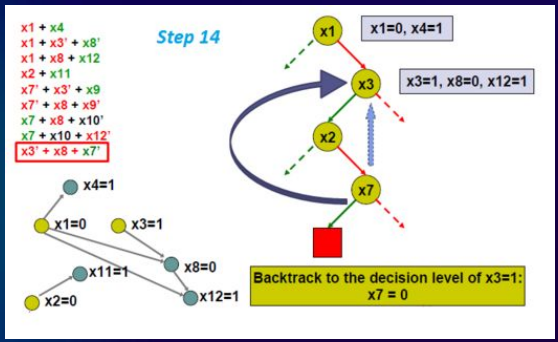
CDCL (Conflict-driven clause learning) 演算法











摘要與簡介

Backtrack 暴力解與傳統求解器

我們透過分析 LSU 深層結構並且自己撰寫基於 backtrack 的暴力解算法來進一步瞭解 MaxSAT，以下使用 **Literal 表示變數**，**Assignment 表示指派**，並以 **Clauses 表示子句**，backtrack 算法詳細如下：

1. 產生一段由各 Literals 組成的初始隨機 Assignment，以 [0, 0, 1, 0, ...] 表示，第 i 位為 0 代表第 i Literal 為 False，反之為 True。
2. 實際帶入該 Assignment 到問題內解解看能不能得到整體結果為 True，若可以則找到一個解，反之繼續第三部。
3. 呼叫 backtrack 並傳遞 i 和修改第 0 Literal 為 False 後的 assignment，在此 backtrack 完成回到原 function 後再度呼叫 backtrack 並傳遞 i 和修改第 0 Literal 為 True 後的 assignemnt 來解第 0 Literal 為 True 的另一種情況。
4. 在暴力解中嘗試透過更改每個位置的 Literal 並生成更多不同的 Assignments 來檢查他們是否滿足此問題的解

摘要與簡介

Backtrack 暴力解

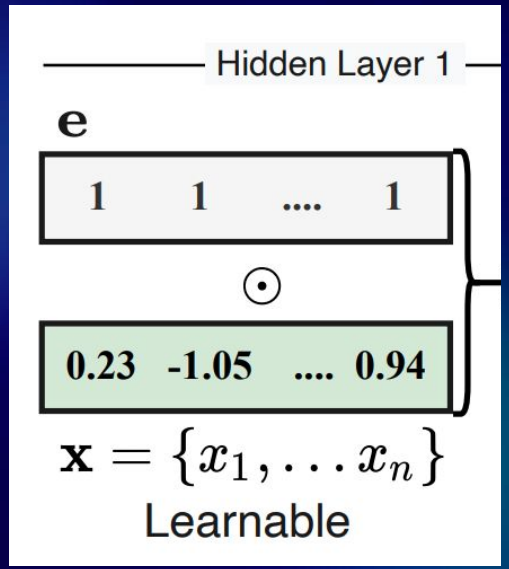
```python
def _backtrack(
    self,
    i: int,
    current_assignments: List[bool],
    assign: bool,
):
  if i == len(obj= self.active_variables):
    current_sat_count = 0
    for clause in self.clauses:
      is_sat_clause = False
      for variable in clause:
        if (variable < 0 and not current_assignments[abs(x= variable)]) \
        or (variable > 0 and current_assignments[abs(x= variable)]):
            is_sat_clause = True
            break
      if is_sat_clause:
        current_sat_count += 1

    if current_sat_count > self.sat_count:
      self.assignments[:] = current_assignments
      self.sat_count = current_sat_count
    return

  current_assignments[self.active_variables[i]] = assign

  self._backtrack(i= i + 1, current_assignments= current_assignments[:], assign= True)
  self._backtrack(i= i + 1, current_assignments= current_assignments[:], assign= False)
```

```python
def solve(self) -> bool:
  self.reduce_variable_space()
  init_assignments = self.assignments.copy()
  self._backtrack(
    i= 1,
    current_assignments= init_assignments,
    assign= True,
  )
  self._backtrack(
    i= 1,
    current_assignments= init_assignments,
    assign= False,
  )
  for clause in self.clauses:
    for variable in clause:
      if (variable < 0 and not self.assignments[abs(x= variable)]) \
      or (variable ≥ 0 and self.assignments[abs(x= variable)]):
        self.answer_of_clauses.append(object= True)
        break
```

左圖為 backtrack 遞迴程式，右為初始呼叫的進入點，由於 MaxSAT 問題使用生成的測資，所以先呼叫 reduce_variable_space() 簡化並確保沒有空 clauses。

研究動機

TorchmSAT



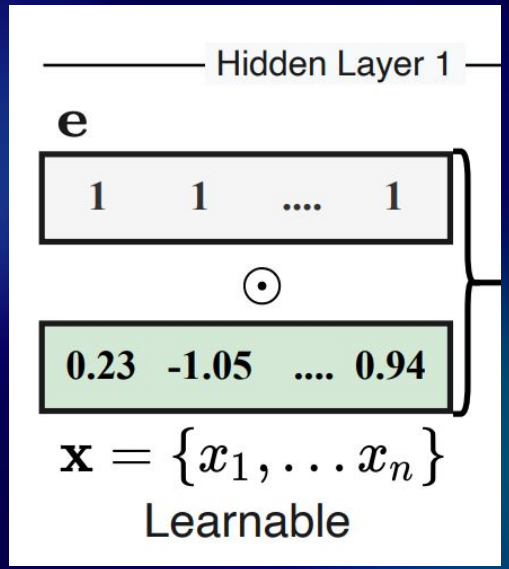
關於向量 e：
第一個隱藏層（ First Hidden Layer ）會對 x 進行逐元素（ point-wise ）乘法：

$$\mathbf{x} \odot \mathbf{e}$$

其中，e 是**與 x 相同長度的向量**，但**所有值都設為 1**。
在反向傳播時，梯度是透過鏈式法則 (Chain Rule) 來計算的。如果沒有 e，梯度可能會被矩陣運算「壓縮」或「改變形狀」，導致梯度回傳時無法對應回 x 的所有分量，因此需要這個 向量 e 來確保 x 的每一個元素都能夠接收到來自 f(x) 的梯度。

研究動機

TorchmSAT



關於向量 x:

為了**逆轉鬆弛（ Reverse the Relaxation ）**，我們採用如下的方式來解釋變數 x_i ：

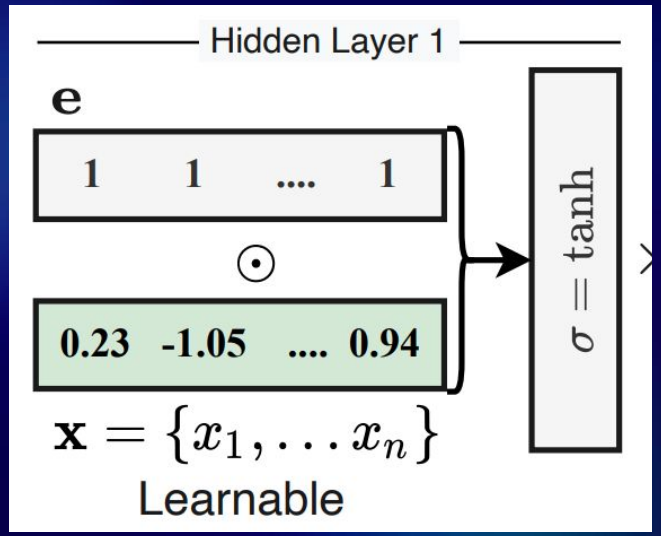
- If $x_i > 0$, then $x_i = 1$（ True ）
- If $x_i <= 0$, then $x_i = 0$（ False ）

變數 x 會被**初始化為隨機的實數 值**。此時，直接解釋這些數 值相當於隨機地給布林變數指派 True 或 False。

在求解過程中， x 的數值會逐步更新，這樣的設計得以更好的應用**最小化損失函數 （ loss function ）**來做 backpropagation。

研究動機

TorchmSAT



為了穩定反向傳播（ Stabilize Backpropagation ），應用 **tanh（雙曲正切）激活函數 （ activate function ）**：

tanh (x)

這樣可以將第一層的輸出限制在 [-1, 1] 的範圍內，防止數值過大或過小（數值爆炸或梯度消失 ），避免訓練過程（即求解過程）**過早終止（ Learning Prematurely Stops）**。

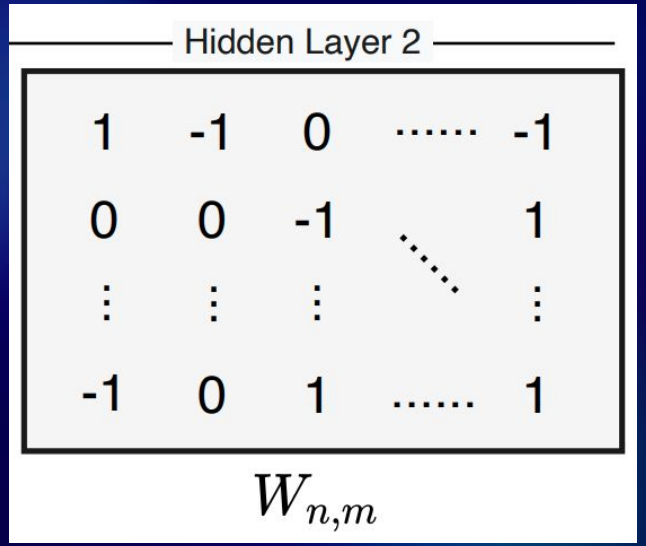
$$\sinh x = \frac{e^x - e^{-x}}{2} \quad \tanh x = \frac{\sinh x}{\cosh x} = \frac{e^x - e^{-x}}{e^x + e^{-x}} = \frac{e^{2x} - 1}{e^{2x} + 1}.$$

$$\cosh x = \frac{e^x + e^{-x}}{2}$$

tanh(x)的定義域為(-inf, inf), 值域為[-1, 1], 且tanh(0) = 0。

研究動機

TorchmSAT



第二個隱藏層（Second Hidden Layer）會包含一個輸入的 MaxSAT 實例（問題本身），為一個 W(n, m)，其中 row（n）代表的是變數，column（m）代表的是 Clause。

舉例：W (1, 2) 代表的是 x_1 在 Clause 2 中的值，
以 1 代表 true，-1 代表 false，0 代表不存在此 Clause 中。

完整例子：假設一個 MaxSAT 問題的 CNF 格式為：

$$F = (\neg x1) \wedge (\neg x2) \wedge (x1 \vee x2)$$

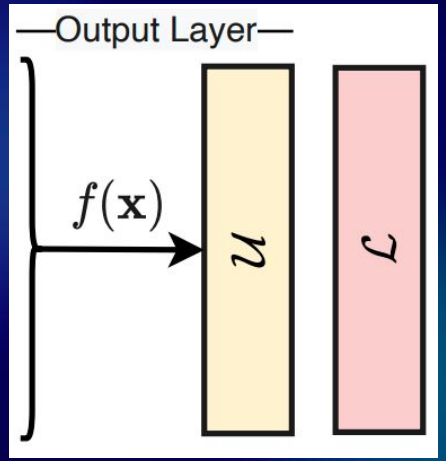
假設 x_1 跟 x_2 為 [illegible] .27]，那在架構中會得到如下公式：

$$f(x) = \tanh\left([1 \quad 1] \odot [0.43 \quad 1.27]\right) \cdot \begin{bmatrix} -1 & 0 & 1 \\ 0 & -1 & 1 \end{bmatrix} = [-0.41 \quad -0.85 \quad 1.26]$$

這代表帶入該 Assignment 後會得到第一個 clause 為 false，第二個 clause 為 false，第三個 clause 為 true 的結果。

研究動機

TorchmSAT



Loss function 的部分，TorchmSAT 使用 Mean Squared Error (均方誤差, MSE) 損失函數:

均方誤差(MSE)是機器學習與深度學習中常用的一種 **回歸損失函數**，它的目標是衡量模型預測 值與真實值之間的誤差大小。
數學定義：

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

其中:
- y_i 是第 i 個真實值(ground truth)
- $\hat{y}_i$ 是第 i 個模型預測值(prediction)
- n 是總樣本數
- **計算誤差**：$y_i - \hat{y}_i$ 計算的是預測 值與真實值之間的誤差(殘差)。
- **平方誤差**：$(y_i - \hat{y}_i)^2$ 用平方來放大較大的誤差，確保誤差為非負 值，同時避免正負抵消。
- **取平均 值**:對所有樣本的平方誤差取平均，讓損失 值與樣本數無關，保持穩定。

研究動機

在處理大規模問題實例時，傳統求解器普遍面臨顯著的計算瓶頸。儘管現代求解器（如RC2）在實務上已展現優異效能，然而其方法多仰賴啟發式策略與搜尋技巧，對於問題內在結構的深層特徵缺乏有效學習能力。另一方面，現有基於神經網路的方法，通常僅採用簡單的訊息聚合機制，未能充分考量不同子句對變數影響程度之差異，因而限制了模型對複雜關聯性的表達能力。

基於上述觀察，本研究旨在引入圖注意力網路Graph Attention Networks, GAT），並透過注意力機制對不同鄰接節點賦予動態權重，建立更精細且具區別性的訊息傳遞流程。藉此提升模型對變數與子句之間複雜依賴關係的建模能力，進一步強化整體求解效能與泛化能力。

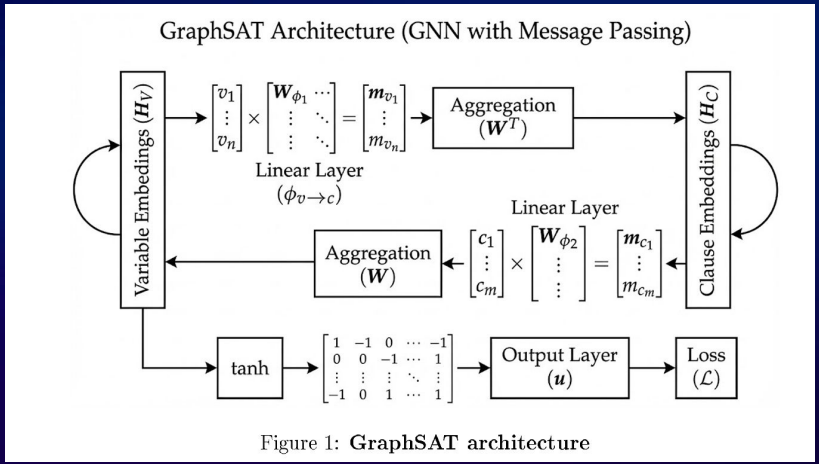


Figure 1: **GraphSAT architecture**

核心理論

理論推導

問題定義：給定一個包含 n 個變數與 m 個子句的 CNF 公式，目標是找到一組變數賦值，使得滿足的子句數量最大化。

基本概念：我們構建一個二分圖G=(V, C, E)，其中 V 為變數，C 為子句。若變數出現在子句中則存在邊。結構由權重鄰接矩陣$W \in R^{n \times m}$ 編碼，其中 1, −1, 0 分別代表正文字、負文字或無連接。

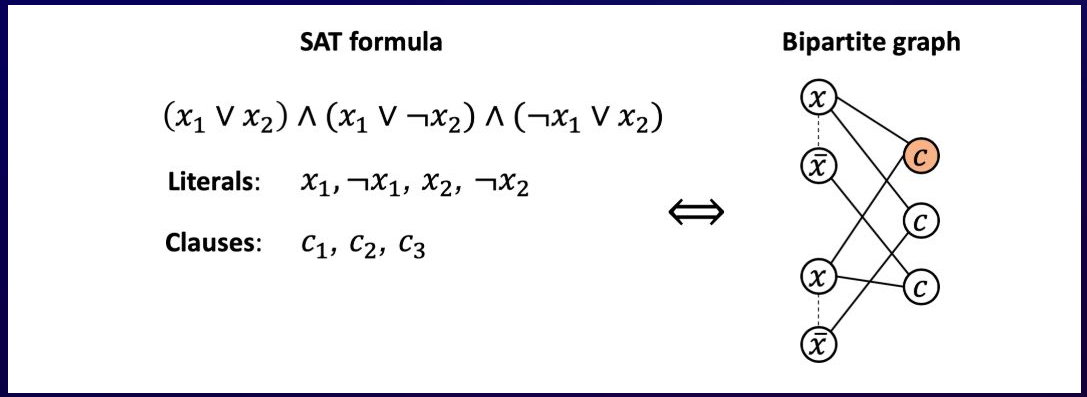
發現與想法：我們發現，在訊息傳遞過程中，不應僅是靜態的鄰居狀態加總，而應是可學習的變換過程。此外，我們還引入Attention 機制允許變數動態關注那些當前未滿足或結構上更重要的子句。

核心理論

架構與演算法則

我們觀察到，相較於傳統的 GraphSAT 一般圖神經網路（GNN）方法，多半著重於同質節點之間的訊息傳遞（例如變數對變數、子句對子句的聚合），因而較難直接捕捉變數與子句之間的跨類型依賴關係。

因此如果讓訊息傳遞的關係涉及變數跟子句之間，讓它們可以彼此互相溝通，模型就能 夠更加理解整個問題的結構進而得到更好的近似解。



GrapghAttentionSAT（Cross Attention）

核心理論

架構與演算法則

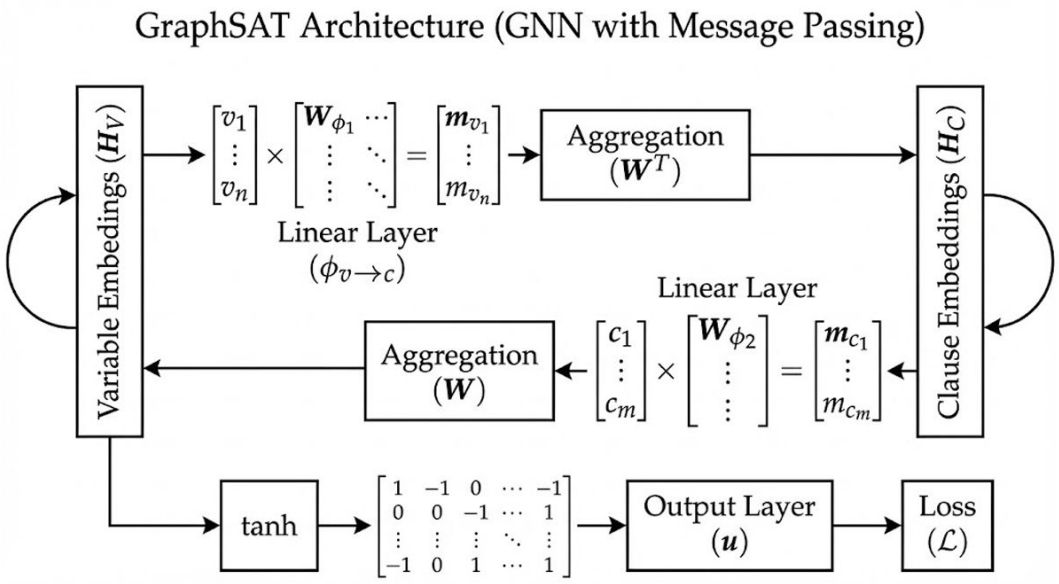


Figure 1: **GraphSAT architecture**

變數到子句的訊息 *Mc*
與子句到變數的訊息 *Mv* ,
分別經過線性變換 ϕv→c 與
ϕc→v 處理:

$MC = W^T \cdot \tanh(\phi v\to c(Hv))$

$MV = W \cdot \tanh(\phi c\to v(Hc))$

這使得模型能學習發送何種
相關資訊。

GrapghSAT Architecture (GNN with message Passing)

核心理論

```python
def __init__(self, nv, clauses, hidden_dim=64):
    super().__init__()
    device = torch.device(device= "cuda") if torch.cuda.is_available() else torch.device(device= "cpu")
    self.device = device
    self.nv = nv
    self.nc = len(obj= clauses)
    W = torch.zeros(size= (nv, self.nc), device=device)
    for j, clause in enumerate(iterable= clauses):
        for lit in clause:
            i = abs(x= lit)-1
            W[i,j] = 1.0 if lit>0 else -1.0
    self.W = W
    self.h_v = nn.Parameter(data= torch.randn(size= (nv, hidden_dim), device=device))
    self.h_c = torch.zeros(size= (self.nc, hidden_dim), device=device)
    self.var_to_clause = nn.Linear(in_features= hidden_dim, out_features= hidden_dim, bias=False, device=device)
    self.clause_to_var = nn.Linear(in_features= hidden_dim, out_features= hidden_dim, bias=False, device=device)
def forward(self, steps=3):
    h_v = self.h_v
    h_c = self.h_c
    for _ in range(stop= steps):
        # messages from variable to clauses
        msg_c = torch.matmul(input= self.W.T, other= torch.tanh(input= self.var_to_clause(h_v))) / (self.W.T.abs().sum(dim=1, keepdim=True)+1e-6)
        h_c = torch.tanh(input= msg_c)
        # messages from clauses to variables
        msg_v = torch.matmul(input= self.W, other= torch.tanh(input= self.clause_to_var(h_c))) / (self.W.abs().sum(dim=1, keepdim=True)+1e-6)
        h_v = torch.tanh(input= h_v + msg_v)
    sol = torch.sign(input= h_v.sum(dim=1, keepdim=True))
    return sol.squeeze()
```


GrapghSAT Architecture (GNN with message Passing)

核心理論

架構與演算法則

基於此，我們試著引入 **Cross-Attention** 架構，以顯式建模變數與子句之間的交互關係，並透過注意力權重進行差異化建模，使整體表示能力在表達力與解釋性上均獲得提升，
其方法為將變數節點視為 查詢向量 **Query,**（ ***Q*** ）而子句節點則對應為鍵 **Key** (***K***) 與值 **Value** (***V***)。透過此設計，使模型能直接建模變數與子句之間的交互關係。
另外，我們引入遮罩機制（ **Mask** ），以確保僅計算相連節點的注意力分數（即變數確實出現在該子句中 ），其公式如下：

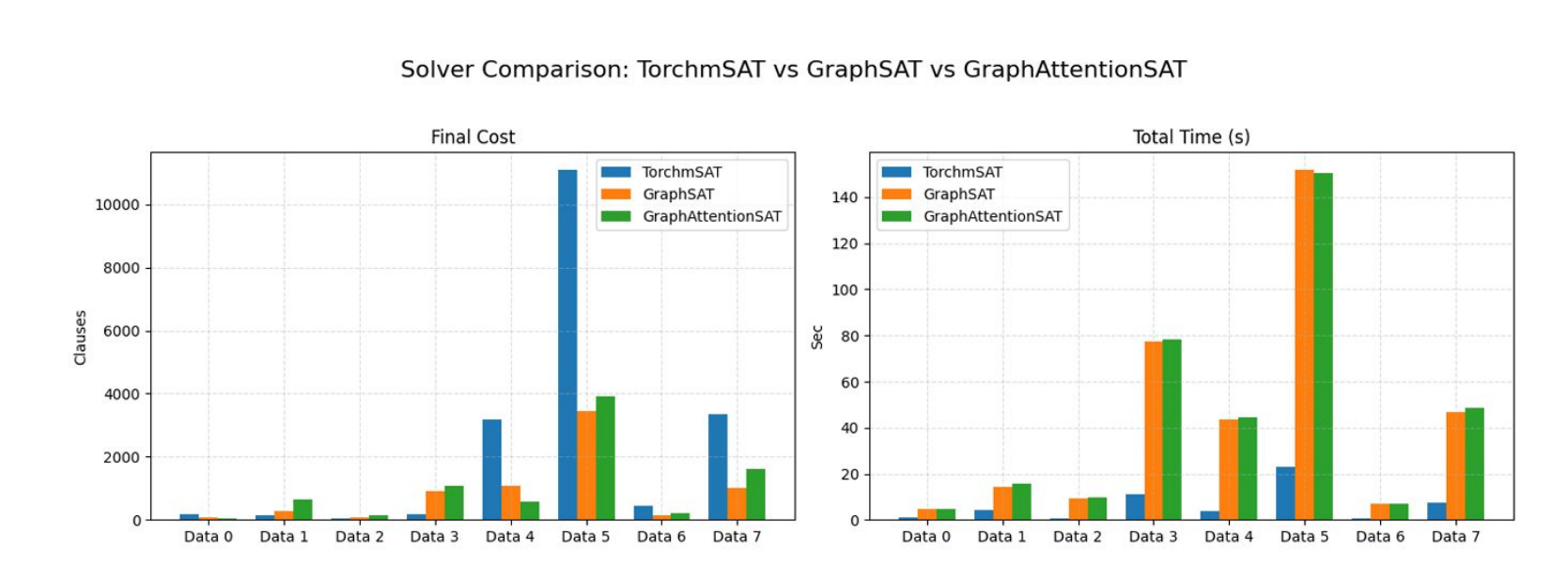
$$\text{Score} = (Qv * Kc^{T}) \ / \ \sqrt{dk}$$

$$\text{Attention} (Q,K,V) = \text{softmax} (\text{Mask}(\text{Score})) \ Vc$$

其中，***dk*** 為鍵向量維度，用於進行尺度縮放（ scaling），以穩定訓練過程。
此外，此機制能 夠根據當前狀態，動態調整不同子句對變數的重要性權重，特別強化對關鍵子句（例如尚未滿足或對整體解影響較大的子句）的關注，從而有效引導求解過程跳 脫局部最佳解。

實驗結果

性能分析與比較



Solver Comparison The final cost is the number of clauses that is unsatisfied on the left. The total time is the execution time.

結論與展望

研究摘要與未來展望

本研究成功將 Max-SAT 問題轉化為一個基於圖神經網路之最佳化問題。透過顯式建構變數與子句之二分圖表示，並結合可學習的訊息傳遞機制，有效克服傳統神經求解方法在結構資訊建模上的不足，提升模型對問題 內在關聯性的理解能力。

在方法表現方面，所提出之結構感知（ structure-aware approach ）方法，相較於原始 torchmSAT 架構，在解的品質與收斂速度上皆展現出顯著提升，驗證了引入結構資訊與注意力機制之有效性。

未來研究將進一步優化 Attension 架構之設計，以更精確地捕捉變數與子句之間的複雜的依賴關係；同時，亦將擴展本方法至更大規模之工業級問題實例，以評估其在實務應用中的可行性與泛化能力。

参考論文

- Hosny, A., & Reda, S. (2024). torchmSAT: A GPU-Accelerated Approximation To The Maximum Satisfiability Problem. arXiv preprint arXiv:2402.03640.
- Neves, M., Martins, R., Janota, M., Lynce, I., & Manquinho, V. (2015). Ex ploiting Resolution-based Representations for MaxSAT Solving. arXiv preprint arXiv:1505.02405.

Thanks!